# Lonestar6 port: task description for the receiving agent

2026-10-09

## What you are being asked to do

Stand up a closed-loop grader-drift experiment on TACC Lonestar6 and run two series of jobs on A100 nodes. The experiment already runs on another cluster (University of Utah CHPC); Lonestar6 is wanted because it has far more GPU capacity. Nothing here is research design — the design is fixed. Your job is the port: build two Python environments, stage the code and data, verify one arm end to end, then launch.

**Do not change experiment parameters** (rounds, intensities, learning rate, epochs, beta) except the three adaptations listed under "Required changes". Those parameters are measured quantities in a running study and changing them silently invalidates comparisons against runs already finished.

---

## 1. The system you are on

**Allocation**: `CDA26011`, 50,000 SUs, expires 2027-09-30. Every job must charge it (`#SBATCH -A CDA26011`).

**GPU hardware**: 84 nodes with 3x NVIDIA A100 PCIe **40 GB** each, 2x AMD EPYC 7763 (128 cores), 256 GB RAM. Also 4 H100 nodes (2x 80 GB) — ignore those unless A100 proves too small.

**Queues** (from `qlimits`):

| queue | max nodes | max wall | max nodes/user | max jobs/user | max submit |
|---|---|---|---|---|---|
| `gpu-a100` | 8 | **48 h** | 12 | 8 | 32 |
| `gpu-a100-small` | 1 | 48 h | 3 | 3 | 12 |
| `gpu-a100-dev` | 2 | 2 h | 2 | 1 | 3 |
| `gpu-h100` | 1 | 48 h | 1 | 1 | 4 |

`sinfo` reports GRES as null, which means **nodes are allocated whole, not per GPU**. A job on `gpu-a100` gets all 3 A100s on its node. Our workload is single-GPU, so a naive port wastes two thirds of every node — see "Packing" below.

**Storage**: `$HOME` 10 GB (too small for anything here), `$WORK` 1 TB (use this), `$SCRATCH` unquota'd but **purged after 10 days of no access** — do not put run outputs there, runs take ~2 days and are analysed for weeks afterwards.

**Modules**: `python/3.12.11`, `python3/3.9.7`, `cuda/11.3 11.4 12.0 12.2 12.8`. **There is no conda module.** You must install your own Miniconda under `$WORK`.

---

# 2. Getting the code and data

## 2.1 Code — ASK THE USER, DO NOT CLONE

The GitHub remote is `https://github.com/Zarzard/DriftedGrader.git`, but **the entire `armorm/` directory is untracked and is not on GitHub**. Cloning gives you a repo without the experiment.

Ask the user to transfer the following from CHPC (`/uufs/chpc.utah.edu/common/home/u1520755/DriftedGrader/armorm/`). This is the minimum set:

```
stage1.py              the experiment driver -- everything runs through this
uf_gold.py             loads the UltraFeedback prompt pool
vintage_shape.py       helpers imported by the above
vintage_analyze.py     helpers imported by the above
s1gsm12.slurm          the GSM8K launcher (adapt this; it is the newest and cleanest)
s1uf18a.slurm          the UltraFeedback launcher
```

If `stage1.py` fails on an import you do not recognise, ask for the named file rather than guessing — several small modules sit next to it.

## 2.2 The `pylibs` shim — REQUIRED, EASY TO MISS

`peft` is **not** installed by pip in the scoring environment. It is supplied by a directory added to `PYTHONPATH`:

`/uufs/chpc.utah.edu/common/home/dbrown-group1/zifan/graderdrift_runs/pylibs  (31 MB)`

You can either copy that directory to `$WORK` and keep using `PYTHONPATH`, or simply `pip install peft==0.13.2` into the scoring environment and drop the `PYTHONPATH` entry. Installing it properly is cleaner. Verify with `python -c "import peft; print(peft.__version__)"` -> `0.13.2`.

## 2.3 Data

**UltraFeedback prompt pool** — copy these two from CHPC (11 MB total), there is no download:

```
graderdrift_runs/bon_pilot/uf_armorm.npz              (0.5 MB)
graderdrift_runs/bon_pilot/uf_armorm_responses.json   (10.4 MB)
```

`uf_gold.py` has a hardcoded path constant `POOL` near line 39 pointing at the CHPC location. **Edit it to your `$WORK` path.** This is one of the required changes.

**GSM8K** downloads from HuggingFace automatically: `openai/gsm8k`, config `main`.

**Models** — four, roughly 60 GB total, all public on HuggingFace:

| role | HF id | note |
|---|---|---|
| policy | `RLHFlow/LLaMA3-SFT` | 8B |
| grader | `nicolinho/QRM-Llama3.1-8B` | 8B, loaded in **fp32** |
| drifted grader | `nicolinho/QRM-Llama3.1-8B-v2` | 8B |
| gold (UF only) | `RLHFlow/ArmoRM-Llama3-8B-v0.1` | 8B |

Set `HF_HOME` to a `$WORK` subdirectory before downloading. Pre-download on a login node with `huggingface-cli download <id>`; compute nodes may not have outbound network, so **do not rely on lazy download inside a job**. After downloading, set `HF_HUB_OFFLINE=1` and `HF_DATASETS_OFFLINE=1` in job scripts.

---

# 3. The two environments

Two separate environments are required. They cannot be merged: generation needs a recent vLLM and torch, scoring needs an older torch that the reward-model code depends on.

### 3.1 `bpref` — scoring, grading, DPO training

```
python 3.8          (3.10 is acceptable if 3.8 is unavailable; test carefully)
torch 2.4.1         CUDA 12.x build
transformers 4.45.2
accelerate 1.0.1
datasets 2.21.0
numpy 1.24.3
scipy 1.10.1
safetensors 0.5.3
peft 0.13.2         (see 2.2)
sentencepiece
```

`stage1.py` asserts `import sentencepiece` succeeds at startup — if it is missing the job exits with a clear message.

### 3.2 `vllmgen` — generation only

```
python 3.11
torch 2.13.0        CUDA 13.0 build
vllm 0.30.0
transformers 5.17.0
numpy 2.3.5
```

**Architecture check.** A100 is compute capability **sm_80**, which both environments support natively. Verify after install:

```
python -c "import torch; print(torch.cuda.get_arch_list())"
```

`sm_80` must appear in both. (Note: on CHPC these environments also run fine on sm_89 hardware via PTX JIT, so a missing exact match is not necessarily fatal — but sm_80 should be present outright.)

---

# 4. Required changes (exactly three, plus the queue block)

**(a) `gpu_memory_utilization` 0.90 -> 0.80.** The experiment runs on 48 GB GPUs elsewhere; the A100 PCIe is **40 GB**. At 0.90 vLLM will try to claim ~43 GB and fail to start. Find the setting in `stage1.py` and lower it. This shrinks the KV cache and will slow generation — that is expected and is what the probe in section 5 measures.

**(b) `uf_gold.py` `POOL` path** -> your `$WORK` copy of `uf_armorm.npz` (see 2.3).

**(c) Output directory.** The slurm scripts set `D=$Z/graderdrift_runs/stage1` where `$Z` is a CHPC path. Point `$Z` at `$WORK`. Each 12-round arm writes ~1.9 GB of LoRA adapters; a full campaign is ~130 GB, which fits the 1 TB quota. **Not `$SCRATCH`** — it purges at 10 days.

**(d) The queue block.** Replace the CHPC `#SBATCH` header with Lonestar6's. Note TACC does not use `--gres` here; the partition implies the GPU and nodes come whole.

---

## 5. First task: the timing probe (do this before anything else)

Run **one** GSM8K arm and measure two numbers. Everything downstream depends on them.

Use `gpu-a100-dev` (2 h limit) for a first smoke test that the stack starts at all, then `gpu-a100-small` for a real timed run of 2-3 rounds.

```
stage1.py --task gsm8k --arm stationary \
  --policy RLHFlow/LLaMA3-SFT --policy-step dpo --reward-model none \
  --grader qrm_v1 --grader-drifted qrm_v2 --drift-round 2 \
  --rounds 3 --n-train 3000 --n-eval 500 --n-samples 8 \
  --max-new 400 --batch 8 --epochs 3 --accum 64 --n-shot 8 \
  --dpo-beta 0.6 --gen-backend vllm --out $WORK/runs/probe_stationary
```

**Report back:**

1. **Hours per round.** On CHPC this is 2.97 h (a6000, UltraFeedback) and 3.90 h (RTX 6000 Ada, GSM8K). A100 should be faster, but the reduced KV cache may offset that. This number decides whether the campaign fits the 48 h wall.
2. **Peak GPU memory**, to confirm 0.80 was the right call and whether it can go higher.
3. **Round-0 held-out accuracy.** Expect **0.76-0.79**. Anything far from that means the model, prompt format, or few-shot setup is wrong — stop and report rather than scaling up.

Sanity values from CHPC for the same configuration: round 0 accuracy 0.758-0.784, rising to ~0.84 by round 3 and holding.

---

## 6. The 48-hour wall is the main obstacle

At CHPC pace:

| series | rounds | time needed | fits 48 h? |
|---|---|---|---|
| GSM8K | 12 | 47.7 h | barely, 0.3 h spare |
| UltraFeedback | 18 | 51.8 h | **no** |
| UltraFeedback | 12 | 34.9 h | yes |

The runs have **no resume by design**. The endogenous drift mechanisms accumulate internal state (`_DRIFT_STATE` in `stage1.py`: a running average `r`, a previous profile `prev`, a cumulative basis `cum`) that no saved adapter reconstructs, so a job killed at the wall loses the entire arm, not just the last round.

Three options, in the order I would try them:

1. **Reduce rounds to fit.** 15 rounds is 43.4 h at CHPC pace. Cheapest, loses the tail.
2. **Make the drift state resumable.** Save `_DRIFT_STATE` next to each round's adapter and reload it with `--resume`. This is a small change but it modifies the thing being measured, so it needs a parity check: one arm run straight through versus the same arm interrupted and resumed must produce identical per-round numbers.
3. **Split each arm across two chained jobs** with `--dependency=afterok`, which still needs (2).

**Decide this with the user after the probe**, because if A100 is appreciably faster than the CHPC pace the problem may disappear for 15 or 18 rounds.

---

## 7. Packing: three arms per node

Nodes come whole with 3 A100s. Running one single-GPU arm per job wastes two thirds of the allocation and burns SUs for nothing.

Write the launcher so one job starts **three** arms on its node, each pinned to a different GPU with `CUDA_VISIBLE_DEVICES=0`, `1`, `2`, backgrounded, with a `wait` at the end. Check that three concurrent arms do not contend badly for the 128 cores and 256 GB — each needs ~28 GB of host RAM, so 84 GB of 256 GB, which is comfortable.

With `gpu-a100` allowing 8 concurrent jobs, packed 3-per-node that is **24 arms at once**, against about 6 on CHPC.

---

## 8. What to run once the probe passes

Ask the user which campaign to start; do not assume. The two candidates are:

**GSM8K, 7 arms, 12 rounds.** Arms: `stationary`, `stationary` (replicate), `matched_random --drift-deg 89`, `matched_random --drift-deg 105`, `momentum --drift-eta 0.50`, `satiation --drift-eta 0.40`, `oracle`. All with `--dpo-beta 0.6`. This is the higher-value series: GSM8K is the only task in the project where gold is a verifiable answer key rather than another model's opinion.

**UltraFeedback replication, 46 runs.** 23 distinct configurations at two fresh seeds each, to put error bars on arms that currently have none. Needs `--gold-model armorm` and the UF pool from 2.3.

---

## 9. Traps that have already cost time on this project

**`--dpo-beta` must be 0.6 on GSM8K.** At the old default of 0.1 the policy's mean output length runs away — 664 to 730 to 1946 characters in one round — generations hit the 400-token cap mid-derivation so no answer is emitted, and held-out accuracy collapses from 0.78 to 0.11 at round 3 or 4. Parse rate stays at 1.000 throughout, so the logs look healthy; **length is the only early warning**. Watch `mean_chars` in `history.json` every round. Flat is good; a jump of 2-3x means the run is dead even if accuracy has not fallen yet.

**Do not add `--gold-filter`.** It was tried, it makes the collapse earlier and deeper, and it is not in the current GSM8K configuration.

**`sacct` reporting COMPLETED does not mean the job worked.** Two arms once exited in 9-15 seconds with rc=127 because a hand-edited slurm script dropped its environment block, and sacct recorded success. Always confirm a run produced rounds, not just that it exited.

**Build new launchers by copying a working one** and changing only the arm table. Do not compose one from fragments.

**Check output is non-degenerate before trusting a batch.** For each finished run verify: `history.json` exists, has the expected number of rounds, `mean_chars` is stable, and `parse_rate` is ~1.0.

---

## 10. Definition of done for the port

1. Both environments build and `torch.cuda.get_arch_list()` contains `sm_80`.
2. Four models and GSM8K are cached under `$WORK`, and a job runs with `HF_HUB_OFFLINE=1`.

3. The probe completes 2-3 rounds with round-0 accuracy in 0.76-0.79.
4. Measured hours per round reported, and the 48-hour decision made with the user.
5. A 3-arms-per-node launcher demonstrated on one node.
6. Campaign launched, with a stated plan for verifying each run's output.

Report the probe numbers before launching anything at scale.